



PlaySBC

Product and Administration Guide

Features, architecture, deployment, operations, regression, and evidence

Version 2.6.0

Final public MIT engineering baseline

Contributor

Sudheer Kumar Vatrappu

Contents

Contents	2
Document Control	4
Using This Guide	4
Opening Regression Evidence	4
Restored Runbook: docs/AI_VOICE_GATEWAY.md	6
PlaySBC AI Voice Gateway	7
Component Roles	7
Rasa Regression Profiles	7
Run The Focused Suite	8
Configuration	8
Evidence Contract	8
v3.0.0 Production Target	8
Restored Runbook: docs/AZURE_AKS.md	10
PlaySBC On Azure AKS	11
Lab Shape	11
1. First-Time Azure Setup	11
2. Deploy Or Upgrade	13
3. Pending LoadBalancer	14
4. Health Check	15
5. Run AKS Regression	15
6. Evidence	17
7. Credential Recovery	17
8. Cleanup	18
Boundaries	19
Restored Runbook: docs/EVOLUTION_PLAN.md	20
PlaySBC Evolution Plan	21
Current Baseline	21
Signalling	21
Media	21
AI Voice Gateway	21
Platform	21
v2.6.0 Local Upgrade Hotfix	21
v2.5.3 TLS Startup Hotfix	21
v2.5.2 Delivered Scope	22
Priority 0: Local Kind Real Devices	22
Evidence Caveats Closed	22
Next Architecture: Local Multi-Node HA Lab	22
Priority 0	22

Acceptance Gates	23
Scope Boundary	23
Near-Term Work	23
HA And Networking	23
Real Devices	23
AI Voice Gateway	23
v3.0.0 AI Voice Gateway Target	23
Production Cloud Track	24
Production Readiness Gates	24
Delivery Rule	24
Restored Runbook: docs/KUBERNETES_HELM_RUNBOOK.md	25
PlaySBC Kubernetes And Helm Runbook	26
Standard Lab	26
Prerequisites	26
Resume The Local Lab	26
Dedicated Local Real-Device Lab	27
Release Upgrade And Full Regression	29
Build Current Source	30
Focused Runs	31
Observe A Run	31
Grafana And Prometheus	31
Debug	32
Cleanup	32
Rules	32
Restored Runbook: docs/KUBERNETES_LOCAL.md	33
Local Kubernetes Lab	34
Default Topology	34
Realm Model	34
kind And minikube	34
Shared State	35
Upcoming Multi-Node HA Lab	35
Local Real Devices	35
Boundaries	35
Restored Runbook: docs/OBSERVABILITY.md	37
PlaySBC Observability	38
What Is Measured	38
Open Grafana	38
Query Prometheus	38
Interpret The Panels	39
Direct Metrics Check	39

Optional Operator Resources	39
Current Boundary	39
Restored Runbook: docs/README.md	40
PlaySBC Documentation	41
Choose A Workflow	41
Default Test Strategy	41
Documentation Rules	41
Restored Runbook: docs/REAL_DEVICE_LAB.md	42
PlaySBC Real-Device Lab	43
1. Apply Real-Device Values	43
2. Configure OBi1022 As 1001	44
3. Configure Zoiper As 1002	45
4. Monitor Signalling And Media	45
5. Capture One Evidence Archive	45
6. Interpret Evidence	46
7. Fast Troubleshooting	47
8. Validated Result And Next Work	47
Restored Runbook: docs/RTPENGINE_LOCAL.md	48
RTPengine For PlaySBC	49
Supported Lab Models	49
Docker Regression	49
Active-Active Pairing	49
Standalone Container	50
Evidence And Diagnosis	50
AKS Complete Cleanup - One Command	51

Document Control

Field	Value
Product	PlaySBC
Release	v2.6.0
Command baseline	Restored canonical AKS, Kubernetes, real-device, media, and operations runbooks
AKS topology	One PlaySBC pod and one RTPengine pod by default
Command policy	Command blocks are reproduced verbatim from the tagged Markdown runbooks

Using This Guide

The browser edition provides working COPY buttons. PDF viewers keep command text selectable but cannot write to the clipboard. Each chapter below identifies its original tagged Markdown source.

Opening Regression Evidence

Every generated regression report now packages its linked artifacts in an `evidence/` directory beside `latest.html`. Keep the complete report directory together when moving or archiving results; copying `latest.html` alone breaks its relative links.

- Text evidence such as SIP traces, workload logs, Kubernetes snapshots, values YAML, and JSON opens in a browser-safe HTML viewer.
- Each viewer provides **Back to report** and **Download raw file** actions.
- Packet captures and other binary evidence open a metadata page with a raw download action for Wireshark or the appropriate desktop application.
- Embedded or linked WAV evidence remains available from AI speech reports.
- A separate localhost server is no longer required for normal viewing. The `tools/serve_regression_report.py` command remains available as an optional presentation mode.

Open a copied Kubernetes regression report directly on macOS:

COMMAND	SELECT + COPY
<pre>open "/absolute/path/to/k8s-reports/latest.html"</pre>	

For the links to remain portable, preserve this structure:

COMMAND	SELECT + COPY
<pre>k8s-reports/ -- latest.html -- <run-id>.html -- <run-id>.json -- evidence/ -- <profile>/ -- log.sip -- log.sip.html -- capture.pcap -- capture.pcap.html</pre>	

Restored Runbook: docs/AI_VOICE_GATEWAY.md

PlaySBC AI Voice Gateway

PlaySBC can answer a SIP call as an AI endpoint, anchor its media through RTPengine, convert speech to text, send the transcript to Rasa, synthesize the response, and preserve the evidence in one report.

COMMAND	SELECT + COPY
SIPp caller -> PlaySBC -> RTPengine -> STT -> Rasa -> TTS -> RTP response	

Component Roles

Component	Responsibility
PlaySBC	SIP/B2BUA control, media conversion, AI orchestration, and evidence
RTPengine	RTP/RTCP anchoring and media transformation
Vosk or Whisper	STT through the shared adapter boundary
Rasa	Intent recognition, dialogue, and bot responses
Piper or Coqui	TTS through the shared adapter boundary
SIPp	Voice traffic and speech-PCAP playback

Rasa Regression Profiles

Run `--rasa-profiles` to execute only the AI/Rasa suite. The report is written to:

COMMAND	SELECT + COPY
logs/RASA-Regression/<run-id>/RASA-reports/latest.html	

Profile	Validates	Primary evidence
ai-rasa-lab	PlaySBC AI route with mock Rasa	SIP/AI logs, ladder, merged PCAP
ai-rasa-rtengine	Mock Rasa with anchored media	RTPengine query and media logs
ai-rasa-real-lab	Real Rasa train/start/webhook path	Rasa rollout, webhook, and SIP evidence
ai-rasa-rtengine-speech	G.711 speech, Vosk, Rasa, and Piper	Input/output WAV, transcript, RTP prompt
ai-rasa-rtengine-speech-whisper	Whisper STT alternative	Provider, transcript, WAV, and RTP evidence
ai-rasa-long-response-streaming	Ordered TTS chunks for long replies	Stream logs and per-chunk artifacts
ai-rasa-contact-center-sales	Vosk/Piper contact-center workflow	Sales ladder and speech evidence
ai-rasa-contact-center-sales-coqui	Coqui TTS alternative	Renderer and generated prompt evidence
ai-rasa-chat-nlu	Positive intent matrix	Chat window, JSON verdicts, NLU ladder
ai-rasa-chat-negative	Guardrails and negative inputs	Guardrail chat window and JSON verdicts

The negative matrix covers denial, ambiguity, empty input, fallback text, special characters, long input, unsupported language, offensive input, and latest-instruction handling.

Case definitions:

COMMAND

SELECT + COPY

```
tests/rasa/chat_nlu_cases.yml
tests/rasa/chat_negative_cases.yml
```

Run The Focused Suite

COMMAND

SELECT + COPY

```
kubectll config use-context kind-playsbc
kubectll config set-context --current --namespace=playsbc

PYTHONPYCACHEPREFIX=/private/tmp/playsbc-pycache \
python3 tools/run_k8s_regression_job.py \
  --rasa-profiles \
  --build-playsbc-image \
  --build-runner-image \
  --build-sipp-image \
  --kind-load-images \
  --kind-cluster playsbc
```

Use KUBERNETES_HELM_RUNBOOK.md for installation, image, observability, and cleanup commands.

Configuration

COMMAND

SELECT + COPY

```
route_policies:
  - name: ai-rasa-gateway
    match: ai-bot
    target: ai-gateway:rasa-support

ai_voice_gateway:
  enabled: true
  provider: rasa
  rasa_webhook_url: http://rasa:5005/webhooks/rest/webhook
  input_mode: speech
  stt_provider: vosk
  tts_provider: piper
  response_mode: rest
```

Adapter alternatives:

COMMAND

SELECT + COPY

```
ai_voice_gateway:
  stt_provider: whisper
  stt_command: python3 tools/whisper_stt_wrapper.py --audio {audio_path} --fallback-transcript "{text}" --allow-lab-fallback
  tts_provider: coqui
  tts_command: python3 tools/coqui_tts_wrapper.py --text "{text}" --output {audio_path} --allow-lab-fallback
  response_mode: streaming
  tts_chunk_chars: 120
```

Evidence Contract

Each voice profile should provide sipmsg.log, one merged capture.pcap, SIP/media/AI logs, an aligned ladder, and playable WAV evidence when speech is involved. Chat profiles provide an initially collapsed chat window, NLU verdict JSON, and an NLP ladder; old voice audio is not shown on chat-only reports.

v3.0.0 Production Target

- Support multiple bot integrations through a stable provider adapter instead of coupling call control to one bot.
- Feed generated TTS RTP into live calls for every provider path and prove both media directions.
- Package production model images and explicit health/readiness contracts for STT and TTS providers.

- Add stateful multi-turn workflows, RFC 4733 DTMF, transfer, conference, fallback, and bot-driven release.
- Export per-provider STT, bot, TTS, streaming, fallback, and action latency/error metrics.
- Preserve canonical SIP/RTP/RTCP/AI evidence and all existing Docker, kind, AKS, and real-device gates.

Restored Runbook: docs/AZURE_AKS.md

PlaySBC On Azure AKS

This is the canonical Azure guide for creating a small AKS lab, deploying PlaySBC/RTPEngine, running AKS regression, downloading evidence, recovering credentials, and deleting the lab. Use REAL_DEVICE_LAB.md only after this base deployment is healthy.

The Azure exposure track was introduced in v1.5.0, and v2.4.0 added strict public-media evidence validation. Every runnable command below targets the current v2.6.0 release.

Keep AKS test windows short. Local kind is the normal lane for full regression and HA/failover iteration.

Lab Shape

COMMAND	SELECT + COPY
<pre>Internet SIP peer -> Azure SIP LoadBalancer -> PlaySBC (1 pod) -> RTPengine (1 pod) -> Azure RTP LoadBalancer, UDP 30000-30049</pre>	

The default AKS readiness lane is intentionally single-workload. Pass explicit HA settings only for a cloud HA milestone.

1. First-Time Azure Setup

Open Azure Cloud Shell in Bash and set stable names:

COMMAND	SELECT + COPY
<pre>export LOCATION=eastus export AKS_RG=playsbc-aks-rg export NETWORK_RG=playsbc-network-rg export AKS_NAME=playsbc-aks export ACR_NAME=playsbcacr\$RANDOM export SIP_PIP_NAME=playsbc-sip-pip export RTP_PIP_NAME=playsbc-rtp-pip export DNS_LABEL=playsbc-sip-lab-\$RANDOM export RTP_DNS_LABEL=playsbc-rtp-lab-\$RANDOM export PLAYSBC_VERSION=2.6.0</pre>	

Cloud Shell variables are ephemeral. Re-export them after reconnecting. For an existing ACR, derive its name instead of generating a new one:

COMMAND	SELECT + COPY
<pre>export ACR_NAME=\$(az acr list --resource-group "\$AKS_RG" --query '[0].name' -o tsv)</pre>	

Register providers once:

COMMAND	SELECT + COPY
<pre>for PROVIDER in \ Microsoft.CloudShell \ Microsoft.ContainerRegistry \ Microsoft.ContainerService \ Microsoft.Network \ Microsoft.Compute \ Microsoft.ManagedIdentity; do az provider register --namespace "\$PROVIDER" done az provider list \ --query "[?contains(namespace, 'Microsoft.Container')] namespace=='Microsoft.Network' namespace=='Microsoft.Compute' namespace=='Microsoft.ManagedIdentity'" \ -o table</pre>	

Continue when the required providers show Registered.

Create resource groups and ACR:

COMMAND	SELECT + COPY
<pre>az group create --name "\$AKS_RG" --location "\$LOCATION" az group create --name "\$NETWORK_RG" --location "\$LOCATION" az acr create \ --resource-group "\$AKS_RG" \ --name "\$ACR_NAME" \ --sku Basic</pre>	

Import all release images:

COMMAND	SELECT + COPY
<pre>for IMAGE in playsbc playsbc-rtppengine playsbc-k8s-regression playsbc-sipp; do az acr import \ --name "\$ACR_NAME" \ --source "ghcr.io/sudheerkumarvatrapu/\$IMAGE:\$PLAYSBC_VERSION" \ --image "\$IMAGE:\$PLAYSBC_VERSION" \ --force done export ACR_LOGIN_SERVER=\$(az acr show \ --resource-group "\$AKS_RG" \ --name "\$ACR_NAME" \ --query loginServer -o tsv) echo "\$ACR_LOGIN_SERVER"</pre>	

Always use Azure's returned ACR_LOGIN_SERVER. Do not construct \$ACR_NAME.azurecr.io; an empty Cloud Shell variable produces InvalidImageName workloads.

Create one-node AKS:

COMMAND	SELECT + COPY
<pre>az aks create \ --resource-group "\$AKS_RG" \ --name "\$AKS_NAME" \ --location "\$LOCATION" \ --tier free \ --node-count 1 \ --node-vm-size Standard_D2as_v7 \ --load-balancer-sku standard \ --attach-acr "\$ACR_NAME" \ --generate-ssh-keys</pre>	

If that VM size is unavailable, choose another two-vCPU size offered by the subscription/region.

Create static SIP and RTP public IPs:

COMMAND	SELECT + COPY
<pre> az network public-ip create \ --resource-group "\$NETWORK_RG" \ --name "\$SIP_PIP_NAME" \ --sku Standard \ --allocation-method static \ --version IPv4 \ --dns-name "\$DNS_LABEL" az network public-ip create \ --resource-group "\$NETWORK_RG" \ --name "\$RTP_PIP_NAME" \ --sku Standard \ --allocation-method static \ --version IPv4 \ --dns-name "\$RTP_DNS_LABEL" </pre>	

Grant both AKS identities permission on the network resource group:

COMMAND	SELECT + COPY
<pre> NETWORK_RG_ID=\$(az group show --name "\$NETWORK_RG" --query id -o tsv) CLUSTER_ID=\$(az aks show -g "\$AKS_RG" -n "\$AKS_NAME" --query identity.principalId -o tsv) KUBELET_ID=\$(az aks show -g "\$AKS_RG" -n "\$AKS_NAME" --query identityProfile.kubeletidentity.objectId -o tsv) for OBJECT_ID in "\$CLUSTER_ID" "\$KUBELET_ID"; do az role assignment create \ --assignee-object-id "\$OBJECT_ID" \ --assignee-principal-type ServicePrincipal \ --role "Network Contributor" \ --scope "\$NETWORK_RG_ID" true done </pre>	

Connect:

COMMAND	SELECT + COPY
<pre> az aks get-credentials \ --resource-group "\$AKS_RG" \ --name "\$AKS_NAME" \ --overwrite-existing kubectl get nodes </pre>	

2. Deploy Or Upgrade

Re-export stable values in every new Cloud Shell session:

COMMAND	SELECT + COPY
<pre> export PLAYSBC_VERSION=2.6.0 export AKS_RG=playsbc-aks-rg export NETWORK_RG=playsbc-network-rg export AKS_NAME=playsbc-aks export SIP_PIP_NAME=playsbc-sip-pip export RTP_PIP_NAME=playsbc-rtp-pip export ACR_NAME=\$(az acr list --resource-group "\$AKS_RG" --query '[0].name' -o tsv) export ACR_LOGIN_SERVER=\$(az acr show -g "\$AKS_RG" -n "\$ACR_NAME" --query loginServer -o tsv) export NODE_RG=\$(az aks show -g "\$AKS_RG" -n "\$AKS_NAME" --query nodeResourceGroup -o tsv) export SIP_PUBLIC_IP=\$(az network public-ip show -g "\$NETWORK_RG" -n "\$SIP_PIP_NAME" --query ipAddress -o tsv) export RTP_PUBLIC_IP=\$(az network public-ip show -g "\$NETWORK_RG" -n "\$RTP_PIP_NAME" --query ipAddress -o tsv) </pre>	

Verify before Helm changes anything:

COMMAND	SELECT + COPY
<pre> : "\${ACR_NAME:?Missing ACR_NAME}" : "\${ACR_LOGIN_SERVER:?Missing ACR_LOGIN_SERVER}" : "\${NODE_RG:?Missing NODE_RG}" : "\${SIP_PUBLIC_IP:?Missing SIP_PUBLIC_IP}" : "\${RTP_PUBLIC_IP:?Missing RTP_PUBLIC_IP}" for IMAGE in playsbc playsbc-rtpengine; do az acr repository show \ --name "\$ACR_NAME" \ --image "\$IMAGE:\$PLAYSBC_VERSION" \ -o none done </pre>	

Deploy atomically so a failed pull cannot replace the healthy revision:

COMMAND	SELECT + COPY
<pre> helm upgrade --install playsbc \ "https://github.com/sudheerkumarvatrapu/PlaySBC/releases/download/v\${PLAYSBC_VERSION}/playsbc-\${PLAYSBC_VERSION}.tgz" \ --namespace playsbc \ --create-namespace \ --reuse-values \ --atomic \ --wait \ --timeout 10m \ --set cloud.provider=azure \ --set cloud.azure.enabled=true \ --set cloud.azure.nodeResourceGroup="\$NODE_RG" \ --set cloud.azure.sip.public.enabled=true \ --set cloud.azure.sip.public.publicIPResourceGroup="\$NETWORK_RG" \ --set cloud.azure.sip.public.publicIPName="\$SIP_PIP_NAME" \ --set cloud.azure.media.public.enabled=true \ --set cloud.azure.media.public.publicIPResourceGroup="\$NETWORK_RG" \ --set cloud.azure.media.public.publicIPName="\$RTP_PIP_NAME" \ --set cloud.azure.media.public.portRange.enabled=true \ --set cloud.azure.media.public.portRange.min=30000 \ --set cloud.azure.media.public.portRange.max=30049 \ --set topology.activeActive.enabled=false \ --set replicaCount=1 \ </pre>	
COMMAND (CONTINUED)	SELECT + COPY
<pre> --set image.repository="\$ACR_LOGIN_SERVER/playsbc" \ --set-string image.tag="\$PLAYSBC_VERSION" \ --set image.pullPolicy=Always \ --set rtpengine.enabled=true \ --set rtpengine.replicas=1 \ --set rtpengine.image.repository="\$ACR_LOGIN_SERVER/playsbc-rtpengine" \ --set-string rtpengine.image.tag="\$PLAYSBC_VERSION" \ --set rtpengine.image.pullPolicy=Always \ --set rtpengine.rtpMin=30000 \ --set rtpengine.rtpMax=30049 \ --set-string rtpengine.advertisedIP="\$RTP_PUBLIC_IP" \ --set playsbc.config.rtp_min=30000 \ --set playsbc.config.rtp_max=30049 \ --set playsbc.config.media_backend=rtpengine \ --set-string playsbc.config.rtpengine_url=udp://playsbc-playsbc-rtpengine:2223 \ --set-string playsbc.config.sip_advertised_ip="\$SIP_PUBLIC_IP" \ --set-string playsbc.config.b2bua_advertised_ip="\$SIP_PUBLIC_IP" </pre>	

Verify:

COMMAND	SELECT + COPY
<pre> kubectl -n playsbc rollout status deployment/playsbc-playsbc --timeout=240s kubectl -n playsbc rollout status deployment/playsbc-playsbc-rtpengine --timeout=240s kubectl -n playsbc get pods -o wide kubectl -n playsbc get svc playsbc-playsbc-azure-sip-public playsbc-playsbc-azure-rtp-public -o wide </pre>	

Do not test until both services have external IPs and they match SIP_PUBLIC_IP and RTP_PUBLIC_IP.

3. Pending LoadBalancer

Inspect events:

COMMAND

SELECT + COPY

```
kubectl -n playsbc describe svc playsbc-playsbc-azure-sip-public | sed -n '/Events:/, $p'
kubectl -n playsbc describe svc playsbc-playsbc-azure-rtp-public | sed -n '/Events:/, $p'
```

- EnsuringLoadBalancer: wait for Azure allocation.
- AuthorizationFailed on Microsoft.Network/publicIPAddresses/read: repeat the Network Contributor assignments for CLUSTER_ID and KUBELET_ID, wait for RBAC propagation, and let the services reconcile.
- Wrong public IP: verify the service annotations reference \$NETWORK_RG, \$SIP_PIP_NAME, and \$RTP_PIP_NAME.

After correcting RBAC:

COMMAND

SELECT + COPY

```
kubectl -n playsbc delete svc \
  playsbc-playsbc-azure-sip-public \
  playsbc-playsbc-azure-rtp-public

helm upgrade playsbc \
  "https://github.com/sudheerkumarvatrapu/PlaySBC/releases/download/v${PLAYSBC_VERSION}/playsbc-${PLAYSBC_VERSION}.tgz" \
  --namespace playsbc \
  --reuse-values

watch -n 5 'kubectl -n playsbc get svc playsbc-playsbc-azure-sip-public playsbc-playsbc-azure-rtp-public -o wide'
```

4. Health Check

COMMAND

SELECT + COPY

```
kubectl -n playsbc exec deployment/playsbc-playsbc -- \
  python3 -c "import urllib.request; print(urllib.request.urlopen('http://127.0.0.1:8080/readyz').read().decode())"

kubectl -n playsbc exec deployment/playsbc-playsbc -- \
  python3 -c "import urllib.request; print(urllib.request.urlopen('http://127.0.0.1:8080/metrics').read().decode()[1000])"
```

Expected: ready and playsbc_active_calls 0.

5. Run AKS Regression

Use the latest main checkout for the Cloud Shell launcher and post-release safety checks. Keep all four runtime images pinned to the immutable release tag; do not use a mutable latest image tag. This gives each run the newest launcher fixes without silently changing the tested PlaySBC, RTPengine, runner, or SIPp version.

Refresh the launcher first. Cloning main creates a normal branch checkout and avoids the harmless detached-HEAD message produced when cloning a release tag:

COMMAND

SELECT + COPY

```
cd ~
rm -rf PlaySBC-main
git clone --branch main --single-branch --depth 1 \
  https://github.com/sudheerkumarvatrapu/PlaySBC.git \
  PlaySBC-main
cd PlaySBC-main
git log --oneline -1
```

Run image import separately so a missing tag is obvious:

COMMAND

SELECT + COPY

```
export PLAYSBC_VERSION=2.6.0
export AKS_RG=playsbc-aks-rg
export ACR_NAME=$(az acr list --resource-group "$AKS_RG" --query '[0].name' -o tsv)
export ACR_LOGIN_SERVER=$(az acr show -g "$AKS_RG" -n "$ACR_NAME" --query loginServer -o tsv)

for IMAGE in playsbc playsbc-rtpengine playsbc-k8s-regression playsbc-sipp; do
  echo "Importing $IMAGE:$PLAYSBC_VERSION"
  az acr import \
    --name "$ACR_NAME" \
    --source "ghcr.io/sudheerkumarvatrapu/$IMAGE:$PLAYSBC_VERSION" \
    --image "$IMAGE:$PLAYSBC_VERSION" \
    --force || break
done
```

Then run from the refreshed checkout:

COMMAND

SELECT + COPY

```
cd PlaySBC-main

PYTHONPYCACHEPREFIX=/tmp/playsbc-pycache \
python3 tools/run_k8s_regression_job.py \
  --aks-profiles \
  --runner-image "$ACR_LOGIN_SERVER/playsbc-k8s-regression:$PLAYSBC_VERSION" \
  --runner-image-pull-policy Always \
  --sipp-image "$ACR_LOGIN_SERVER/playsbc-sipp:$PLAYSBC_VERSION" \
  --sipp-image-pull-policy Always \
  --playsbc-image "$ACR_LOGIN_SERVER/playsbc:$PLAYSBC_VERSION" \
  --rtpengine-image "$ACR_LOGIN_SERVER/playsbc-rtpengine:$PLAYSBC_VERSION" \
  --set-playsbc-image \
  --set-rtpengine-image \
  --aks-load-balancer-wait-timeout 1200 \
  --job-timeout 3600
```

Expected startup output includes the selected release/main commit from `git log`, followed by Launching Azure AKS Regression Job for 12 profiles. The report must show release image tag 2.6.0; this proves the launcher and runtime image contract are aligned.

The `--aks-profiles` shortcut enforces Azure services, static SIP, public SIP/RTP ingress, UDP 30000-30049, and single-workload topology. Image references are validated before Helm or Job mutation.

AKS profile runs default both regression images to `imagePullPolicy: Always`. Release tags are immutable; the pull policy protects explicit main compatibility runs from stale node-cached runner code. Confirm the GHCR workflow and all four ACR imports completed before launching the Job.

For the mixed `tls-srtp-to-udp-rtp` and `udp-rtp-to-tls-srtp` profiles, the runner starts `send_srtp_audio.py` as a separate, tracked process in the secure endpoint pod. The helper advertises a deterministic SDES key, binds before the call, learns RTPengine's source endpoint from the first received packet, and returns authenticated AES-CM/HMAC-SHA1-80 PCMU traffic. Its command, stdout, stderr, timeout, and exit status are first-class regression evidence. SIPp XML contains SDP only; it does not launch shell commands. This is synthetic regression-endpoint behavior and does not alter the real-device Helm media policy.

The secure helper binds reserved one-call profile ports 6000/UDP for RTP and 6001/UDP for RTCP. The same values are explicit in rendered SDP and the runner command. Plain RTP profiles continue using SIPp's dynamic media-port allocation. If encrypted packets reach the secure pod but none return, inspect `core-secure-srtp-sender/` or `peer-secure-srtp-sender/` before investigating Azure networking.

Stop only regression resources:

COMMAND	SELECT + COPY
<pre>kubectl -n playsbc delete job \ -l app.kubernetes.io/name=playsbc-k8s-regression-runner \ --ignore-not-found kubectl -n playsbc delete pod \ -l app.kubernetes.io/name=playsbc-k8s-regression-runner \ --ignore-not-found</pre>	

6. Evidence

COMMAND	SELECT + COPY
<pre>RUN=\$(ls -td ~/PlaySBC-main/logs/AKS-Regression/aks-regression-* head -1) echo "\$RUN" tail -120 "\$RUN/runner.log" ls -lh "\$RUN/AKS-reports/latest.html" ARCHIVE=~/PlaySBC-main/logs/AKS-Regression/latest-aks-regression.tgz ls -lh "\$ARCHIVE" tar -tzf "\$ARCHIVE" head -40</pre>	

Download the .tgz immediately. Cloud Shell storage can be ephemeral.

Each single-call profile keeps one combined capture.pcap, one sipmsg.log, focused logs, and an HTML report. Load profiles may omit PCAP by design.

7. Credential Recovery

Try normal refresh first:

COMMAND	SELECT + COPY
<pre>export AKS_RG=playsbc-aks-rg export AKS_NAME=playsbc-aks az aks get-credentials -g "\$AKS_RG" -n "\$AKS_NAME" --overwrite-existing</pre>	

If Azure CLI returns `InvalidApiVersionParameter`, first verify that the active subscription contains the cluster:

COMMAND	SELECT + COPY
<pre>az account show --query '{subscription:name,id:id}' -o table az aks list --query '[].{name:name,resourceGroup:resourceGroup}' -o table</pre>	

If `playsbc-aks` is absent, select the subscription that owns it and list the clusters again:

COMMAND	SELECT + COPY
<pre>az account list -o table az account set --subscription '<subscription-id-containing-playsbc-aks>' az aks list --query '[].{name:name,resourceGroup:resourceGroup}' -o table</pre>	

Then use the guarded REST fallback. It discovers the resource group from Azure, rejects empty variables, and only replaces kubeconfig after Azure returns a non-empty credential. Do not redirect the REST response straight to `~/ .kube/config`: an Azure error would truncate the working file.

COMMAND	SELECT + COPY
<pre> set -euo pipefail export AKS_NAME=playsbc-aks export AKS_RG=\$(az aks list \ --query "[?name=='playsbc-aks'].resourceGroup" [0]) \ -o tsv) export SUB_ID=\$(az account show --query id -o tsv) : "\${SUB_ID:?Subscription ID is empty}" : "\${AKS_RG:?playsbc-aks was not found in the active subscription}" : "\${AKS_NAME:?AKS name is empty}" echo "SUB_ID=\$SUB_ID" echo "AKS_RG=\$AKS_RG" echo "AKS_NAME=\$AKS_NAME" TMP_KUBECONFIG=\$(mktemp) az rest \ --method post \ --url "https://management.azure.com/subscriptions/\${SUB_ID}/resourceGroups/\${AKS_RG}/providers/Microsoft.ContainerService/managedClusters/\${AKS_NAME}/listClusterUserCredentials" \ --query 'kubeconfigs[0].value' \ </pre>	
COMMAND (CONTINUED)	SELECT + COPY
<pre> -o tsv base64 -d > "\$TMP_KUBECONFIG" test -s "\$TMP_KUBECONFIG" mkdir -p ~/.kube mv "\$TMP_KUBECONFIG" ~/.kube/config chmod 600 ~/.kube/config kubectl config current-context kubectl get nodes kubectl get pods -n playsbc </pre>	

An error URL containing `resourceGroups/providers/.../managedClusters/listClusterUserCredential` means both `$AKS_RG` and `$AKS_NAME` were empty. Re-run the guarded block above; it stops before calling Azure when cluster discovery fails.

8. Cleanup

Download the evidence archive first. Cloud Shell sessions can change tenants or subscriptions, so discover the active subscription and verify the exact PlaySBC groups before deleting anything:

COMMAND	SELECT + COPY
<pre> unset SUB_ID export SUB_ID=\$(az account show --query id -o tsv) export AKS_RG=playsbc-aks-rg export NETWORK_RG=playsbc-network-rg : "\${SUB_ID:?No active Azure subscription}" echo "SUB_ID=\$SUB_ID" az account show -o table az group list \ --subscription "\$SUB_ID" \ --query "[?name=='\$AKS_RG' name=='\$NETWORK_RG'].{Name:name, Location:location}" \ -o table </pre>	

Both named groups must appear in the table. If they do not, stop and select the subscription that owns the AKS lab with `az account list -o table` and `az account set --subscription <id>`.

Submit both asynchronous deletions only after verification:

COMMAND

SELECT + COPY

```
for RG in "$AKS_RG" "$NETWORK_RG"; do
    EXISTS=$(az group exists \
        --subscription "$SUB_ID" \
        --name "$RG")

    echo "$RG exists: $EXISTS"

    if [ "$EXISTS" = "true" ]; then
        az group delete \
            --subscription "$SUB_ID" \
            --name "$RG" \
            --yes \
            --no-wait
    fi
done
```

Monitor until both checks return false; the loop exits automatically:

COMMAND

SELECT + COPY

```
while true; do
    AKS_EXISTS=$(az group exists --subscription "$SUB_ID" --name "$AKS_RG")
    NETWORK_EXISTS=$(az group exists --subscription "$SUB_ID" --name "$NETWORK_RG")

    date
    echo "AKS_RG exists: $AKS_EXISTS"
    echo "NETWORK_RG exists: $NETWORK_EXISTS"

    if [ "$AKS_EXISTS" = "false" ] && [ "$NETWORK_EXISTS" = "false" ]; then
        echo "PlaySBC AKS lab deletion completed."
        break
    fi

    sleep 30
done
```

The AKS-managed MC_playsbc-aks-rg_playsbc-aks-eastus group should disappear with playsbc-aks-rg; do not delete it separately. Keep Azure-managed groups such as NetworkWatcherRG. The lab is not cost-stopped until both PlaySBC groups report false.

Boundaries

- Use local multi-node kind for frequent HA and failover work.
- Use AKS for Azure identity, ACR, public LoadBalancer, static IP, firewall, and real-device milestones.
- Use REAL_DEVICE_LAB.md for OBi1022/Zoiper configuration and packet capture.
- Production still requires external shared state, multi-zone proof, security hardening, capacity baselines, and long soak tests.

Restored Runbook: docs/EVOLUTION_PLAN.md

PlaySBC Evolution Plan

PlaySBC is an enterprise-style SIP/RTP lab and regression platform. It is not yet a production-certified SBC. Production readiness must be earned through measured scale, security, HA, cloud networking, and long-duration validation.

Current Baseline

Signalling

- SIP UDP/TCP/TLS with REGISTER, OPTIONS, INVITE, ACK, CANCEL, BYE, and digest authentication
- Registrar-backed B2BUA routing, trunk groups, route policies, normalization, CAC, health probing, and node draining
- Shared registrar/dialog/B2BUA lab state and active-active node identity
- Pre-call, mid-call, post-call, RTPengine failure, drain, restore, and load-distribution regression profiles

Media

- PCMU/PCMA, transcoding, RFC 4733, RTP/RTCP, and SDES-SRTP/RTP interworking
- RTPengine anchoring, SDP rewrite, public advertised IP, NAT learning, media handover, and packet verdicts
- One combined PCAP, canonical SIP ladder, media classification, and HTML evidence
- Validated two-way OBi1022/Zoiper PCMU calls through AKS

AI Voice Gateway

- Rasa REST, real Rasa pod, chat/NLU matrices, and guardrail profiles
- Vosk/Whisper STT and Piper/Coqui TTS adapter paths
- Real G.711 speech input, WAV evidence, contact-center sales flow, and long-response chunking

Platform

- Docker dual-realm regression
- Helm deployment for kind, minikube, and AKS
- Active-active PlaySBC/RTPengine lab topology
- Prometheus/Grafana with node, realm, SIP, media, codec, transcoding, and AI metrics
- AKS public SIP/RTP LoadBalancers and strict cloud readiness profiles

v2.6.0 Local Upgrade Hotfix

- Use a local-only Recreate rollout so the old pod releases fixed SIP host ports before its replacement is scheduled.
- Preserve RollingUpdate for AKS and standard Deployment models and preserve StatefulSet behavior for active-active topology.
- Remove the single-node FailedScheduling: didn't have free ports upgrade deadlock without changing signalling or media behavior.

v2.5.3 TLS Startup Hotfix

- Make the CA bundle optional when the local real-device lab disables TLS peer verification.
- Preserve strict certificate and private-key requirements for SIP TLS listeners.

- Validate the dedicated kind deployment live with both workloads ready, all 53 host mappings present, and LAN SIP/RTP advertisement aligned.
- Keep AKS, active-active kind, Docker, regression, media, and AI Voice Gateway behavior unchanged.

v2.5.2 Delivered Scope

The release adds an isolated local kind real-device lane so registration, calls, media, and capture iteration do not require an expensive AKS session.

Priority 0: Local Kind Real Devices

- Added stable one-to-one SIP UDP/TCP/TLS and RTP/RTCP port mappings from a dedicated kind cluster to the home LAN.
- Added a guarded single-workload values profile for OBi1022 1001 and Zoiper 1002 registration and calls.
- Reused the combined PCAP, canonical ladder, media verdict, HTML, and downloadable archive evidence contract.
- Preserved AKS real-device values/public-IP behavior and the existing active-active kind - playsbc regression cluster.
- Added repeatable create, upgrade, preflight, monitor, capture, and cleanup commands.
- Added explicit kube-context pinning so local and AKS evidence tools cannot follow the wrong current context.

Evidence Caveats Closed

- Non-fatal SIPp SSL_ERROR_WANT_READ and watchdog notices are removed from final stderr.log; raw diagnostics remain available.
- Previous-container logs are requested only when Kubernetes reports a restart or terminated previous state.
- AKS exposure and RTPengine advertised-IP validation runs again after each profile rollout.
- PlaySBC evidence uses the exact profile start time and scopes call-ID lines to that profile; unfiltered text is retained only when needed as playsbc.raw.log.
- archive-manifest.txt labels its 200-path preview and reconciles pre-manifest and archived member counts.
- Keep the current strict requirement for one merged capture.pcap, one sipmsg.log, complete ladders, bidirectional media verdicts, and zero packet drops.

The v2.5.1 AKS run aks-regression-20260824-172526 remains the 12-profile baseline. v2.5.2 changes evidence clarity and adds a separate local lane; it does not change the validated AKS signalling/media profile catalog.

Next Architecture: Local Multi-Node HA Lab

This follows the local real-device baseline and moves expensive HA/failover iteration from AKS to a repeatable local environment.

COMMAND	SELECT + COPY
<pre>kind control-plane ■■■ worker-1: PlaySBC-0 + RTPengine-0 ■■■ worker-2: PlaySBC-1 + RTPengine-1</pre>	

Priority 0

- Create a reproducible kind configuration with one control-plane and two workers.
- Pin each PlaySBC/RTPengine pair to separate workers with pod anti-affinity.
- Add PodDisruptionBudgets and controlled node drain behavior.
- Provide shared registrar/dialog state that works across workers; SQLite/RWO remains lab-only, so RWX or Redis/PostgreSQL must be evaluated.

- Run all existing HA profiles through the multi-node topology.
- Kill PlaySBC-0, RTPengine-0, and worker-1 during active calls and record recovery behavior.
- Prove active-active load distribution and restored registrar/dialog ownership.
- Add Grafana panels for worker, PlaySBC node, RTPengine pair, drain state, failover count, and recovery time.
- Keep one canonical SIP/RTP/RTCP PCAP and a clear four-node ladder.

Acceptance Gates

- Existing Docker, single-node kind/minikube, AKS, Rasa, and real-device regressions remain green.
- A failed image or missing variable cannot mutate a healthy deployment.
- Pod failure and node drain have deterministic report verdicts.
- Mid-call tests distinguish signalling recovery from true media continuity.
- The runbook can create, test, inspect, and delete the lab without manual repair.

Scope Boundary

Multi-node kind validates Kubernetes scheduling, pod/node disruption, shared state, and application recovery on one Mac. It does not prove Azure zone failure, physical host failure, or production load-balancer behavior. AKS remains the release-milestone lane for those cloud-specific checks.

Near-Term Work

HA And Networking

- Extend restored mid-call handling to CANCEL, re-INVITE, REFER, and transfer flows.
- Promote RTPengine mid-call recovery toward lossless continuity where Sipwise session ownership permits it.
- Add real core/peer interfaces with Multus or another multi-network CNI.
- Define external SIP affinity, health steering, and graceful drain behavior.
- Replace SQLite lab HA state with Redis/PostgreSQL or another replicated backend.

Real Devices

- Validate OBi1022 and future Poly/Yealink devices over TCP and TLS.
- Add longer calls, re-registration during calls, SIP ALG detection, and multi-device scenarios.
- Run local-LAN real-device tests through a dedicated kind cluster with one-to-one SIP and RTP port mappings.
- Add richer RTCP loss/jitter and MOS-style evidence.

AI Voice Gateway

- Return generated TTS RTP into the live call, not only report evidence.
- Add real model images for Whisper and Coqui.
- Add action-server workflows, multi-turn state, DTMF hybrid IVR, transfer, and barge-in.
- Add STT, Rasa, TTS, streaming, fallback, and action latency metrics.

v3.0.0 AI Voice Gateway Target

After the v2.5.x local/AKS compatibility baseline, the main product focus moves to a production-oriented AI Voice Gateway. v3.0.0 targets multiple bot backends behind one provider interface, live bidirectional TTS/RTP, deterministic interruption and fallback behavior, stateful multi-turn calls, transfer/DTMF workflows, provider health/latency metrics, and complete SIP/media/AI evidence. Every AI change remains subject to the same Docker, kind, AKS, and

real-device compatibility gates.

Production Cloud Track

Azure remains the first production reference cloud; AWS follows.

- External shared state for registrar, dialog, transaction, CDR, audit, and billing events
- Production SIP load balancing and affinity for UDP/TCP/TLS
- Certificate lifecycle, secret rotation, SRTP/DTLS-SRTP policy, and firewall controls
- SIP flood, malformed-message, registration storm, OPTIONS storm, INVITE burst, and overload protection
- Multi-AZ deployment, failure injection, backup/restore, upgrade/rollback, and days-long soak runs
- Capacity progression: 10k, 50k, 100k, then 300k registrations; 250, 500, 1000, then 2500 concurrent calls
- Measured CPU, memory, packets per second, RTP sessions, registrations, dialogs, and calls per second

Production Readiness Gates

- No critical signalling, media, HA, security, or evidence caveats
- Full regression passes on local multi-node and cloud reference topologies
- Load and soak tests pass with packet, media, CDR, and observability proof
- Security scans, fuzzing, malformed SIP, dependency, image, and configuration scans pass
- Operators have tested deploy, scale, drain, failover, backup, restore, upgrade, rollback, and incident procedures

Delivery Rule

Every change must preserve the golden rule: Docker regression, local Kubernetes, AKS regression, AI/Rasa, real-device behavior, and other deployment models must not regress. New behavior needs focused tests, clear logs, combined evidence, and an actionable report verdict.

Historical version-by-version milestones remain in [release notes](../release/README.md).

Restored Runbook: docs/KUBERNETES_HELM_RUNBOOK.md

PlaySBC Kubernetes And Helm Runbook

This is the canonical local Kubernetes command guide. Use KUBERNETES_LOCAL.md for topology and networking concepts.

Standard Lab

COMMAND	SELECT + COPY
PlaySBC-0 + RTPengine-0 PlaySBC-1 + RTPengine-1 Prometheus + Grafana Regression Job + temporary SIPp core/peer pods	

Service	Ports
PlaySBC	5062/UDP, 5062/TCP, 5061/TCP, 8080/TCP
RTPengine control	2223/UDP
Grafana	3000/TCP
Prometheus	9090/TCP

Prerequisites

COMMAND	SELECT + COPY
<pre>open -a Docker until docker info >/dev/null 2>&1; do echo "Waiting for Docker Desktop..." sleep 5 done docker info kubectl version --client helm version --short kind version</pre>	

kind uses Docker containers as Kubernetes nodes. Docker Desktop must remain running while the local PlaySBC lab or regression is running. Minikube is a separate compatibility lane and is not required for the canonical kind-play sbc workflow.

Create the default cluster once:

COMMAND	SELECT + COPY
<pre>kind create cluster --name playsbc kubectl config use-context kind-playsbc kubectl config set-context --current --namespace=playsbc</pre>	

Resume The Local Lab

After a Mac reboot or Docker Desktop shutdown, start Docker and reuse the existing kind cluster:

COMMAND	SELECT + COPY
<pre>open -a Docker until docker info >/dev/null 2>&1; do echo "Waiting for Docker Desktop..." sleep 5 done kind get clusters docker ps -a --filter label=io.x-k8s.kind.cluster kubectl config use-context kind-playsbc kubectl config set-context --current --namespace=playsbc kubectl get nodes kubectl get pods -n playsbc</pre>	

If `playsbc-control-plane` exists but is stopped, resume it and verify the API:

COMMAND	SELECT + COPY
<pre>docker start playsbc-control-plane kubectl cluster-info --context kind-playsbc kubectl get pods -n playsbc</pre>	

An error such as `127.0.0.1:<port>: connect: connection refused` means the kubeconfig context exists but the local kind API container is unavailable. Start Docker Desktop first. Recreate the cluster only when `kind get clusters` does not list `playsbc`:

COMMAND	SELECT + COPY
<pre>kind create cluster --name playsbc --wait 180s kubectl config use-context kind-playsbc kubectl create namespace playsbc --dry-run=client -o yaml kubectl apply -f - kubectl config set-context --current --namespace=playsbc</pre>	

Do not run `kind delete cluster` as a restart step; deletion removes the local Kubernetes workloads and requires a fresh Helm deployment.

Dedicated Local Real-Device Lab

This lane is separate from both `kind-playsbc` regression and AKS:

Purpose	Cluster	Context	Topology
Full local regression	playsbc	kind-playsbc	Active-active
LAN OBi/Zoiper calls	playsbc-real-device	kind-playsbc-real-device	One PlaySBC + one RTPengine
Azure validation	playsbc-aks	AKS context	Azure values and LoadBalancers

Start Docker, discover the Mac LAN address, and create the dedicated cluster once. The port mappings are fixed when kind creates the node, so an older cluster with the same name must be recreated.

COMMAND	SELECT + COPY
<pre> cd /Users/sudheerkumar/Documents/Codex/2026-05-18/Mini-Call-Server export PLAYSBC_VERSION=2.6.0 export REAL_DEVICE_CLUSTER=playsbc-real-device export REAL_DEVICE_CONTEXT=kind-playsbc-real-device export LAN_IF=\$(route -n get default awk '/interface:/{print \$2; exit}') export LAN_IP=\$(ipconfig getifaddr "\$LAN_IF") : "\${LAN_IP:?Could not determine the Mac LAN IPv4 address}" open -a Docker until docker info >/dev/null 2>&1; do sleep 5; done if ! kind get clusters grep -qx "\$REAL_DEVICE_CLUSTER"; then kind create cluster \ --name "\$REAL_DEVICE_CLUSTER" \ --config configs/kubernetes/kind-real-device-cluster.yaml \ --wait 180s fi kubectl --context "\$REAL_DEVICE_CONTEXT" create namespace playsbc \ --dry-run=client -o yaml kubectl --context "\$REAL_DEVICE_CONTEXT" apply -f - </pre>	

Create a short-lived lab TLS secret. UDP and TCP calls do not require the phone to trust this certificate; a hardphone TLS test must import or trust the generated CA/certificate.

COMMAND	SELECT + COPY
<pre> TLS_DIR=\$(mktemp -d) cat >"\$TLS_DIR/openssl.cnf" <<EOF [req] distinguished_name=dn x509_extensions=ext prompt=no [dn] CN=\$LAN_IP [ext] subjectAltName=IP:\$LAN_IP,DNS:playsbc.local keyUsage=digitalSignature,keyEncipherment extendedKeyUsage=serverAuth EOF openssl req -x509 -newkey rsa:2048 -nodes -days 30 -sha256 \ -config "\$TLS_DIR/openssl.cnf" \ -keyout "\$TLS_DIR/tls.key" \ -out "\$TLS_DIR/tls.crt" kubectl --context "\$REAL_DEVICE_CONTEXT" -n playsbc create secret tls playsbc-real-device-tls \ --cert "\$TLS_DIR/tls.crt" \ --key "\$TLS_DIR/tls.key" \ </pre>	
COMMAND (CONTINUED)	SELECT + COPY
<pre> --dry-run=client -o yaml \ kubectl --context "\$REAL_DEVICE_CONTEXT" apply -f - </pre>	

The local lab disables peer certificate verification, so this standard TLS Secret only needs `tls.crt` and `tls.key`. A `ca.crt` entry is required only when `playsbc.config.tls_verify_peer=true`.

Install the isolated values profile and advertise the Mac LAN IP on both signalling and media:

The local profile uses a Recreate rollout because both revisions cannot own the same fixed host ports on one kind node. A short signalling interruption during an upgrade is expected; Helm waits for the replacement pod and avoids a host-port scheduling deadlock.

COMMAND	SELECT + COPY
<pre>helm upgrade --install playsbc \ "https://github.com/sudheerkumarvatrapu/PlaySBC/releases/download/v\${PLAYSBC_VERSION}/playsbc-\${PLAYSBC_VERSION}.tgz" \ --kube-context "\$REAL_DEVICE_CONTEXT" \ --namespace playsbc \ --create-namespace \ --atomic --wait --timeout 10m \ -f configs/kubernetes/kind-real-device-values.yaml \ --set-string localRealDevice.lanIPv4="\$LAN_IP" \ --set-string playsbc.config.sip_advertised_ip="\$LAN_IP" \ --set-string playsbc.config.b2bua_advertised_ip="\$LAN_IP" \ --set-string rtpengine.advertisedIP="\$LAN_IP" \ --set-string tls.existingSecret=playsbc-real-device-tls \ --set-string image.tag="\$PLAYSBC_VERSION" \ --set-string rtpengine.image.tag="\$PLAYSBC_VERSION" kubectl --context "\$REAL_DEVICE_CONTEXT" -n playsbc rollout status \ deployment/playsbc-playsbc --timeout=240s kubectl --context "\$REAL_DEVICE_CONTEXT" -n playsbc rollout status \ deployment/playsbc-playsbc-rtpengine --timeout=240s python3 tools/check_kind_real_device_lab.py \ --context "\$REAL_DEVICE_CONTEXT" \</pre>	
COMMAND (CONTINUED)	SELECT + COPY
<pre>--cluster "\$REAL_DEVICE_CLUSTER" \ --lan-ip "\$LAN_IP" \ --expected-version "\$PLAYSBC_VERSION"</pre>	

Configure both devices with \$LAN_IP, SIP port 5062, users 1001 and 1002, and password secret - password. Monitor and capture without changing the current kube context:

COMMAND	SELECT + COPY
<pre>kubectl --context "\$REAL_DEVICE_CONTEXT" -n playsbc logs -f \ -l app.kubernetes.io/instance=playsbc \ --all-containers=true --prefix --max-log-requests=10 --since=10m \ grep -aE 'REGISTER SIP (INVITE ACK BYE CANCEL) SIP TX response SIP response SDP SUMMARY RTPENGINE RTP packet RTCP 1001 1002' PYTHONPYCACHEPREFIX=/private/tmp/playsbc-pycache \ python3 tools/run_real_device_capture.py \ --context "\$REAL_DEVICE_CONTEXT" \ --namespace playsbc \ --duration 120 \ --capture-image nicolaka/netshoot:latest</pre>	

The capture produces one combined capture.pcap, one sipmsg.log, an HTML report, and one .tgz. Stop this lane without touching kind-playsbc or AKS:

COMMAND	SELECT + COPY
<pre>helm --kube-context "\$REAL_DEVICE_CONTEXT" -n playsbc uninstall playsbc kind delete cluster --name "\$REAL_DEVICE_CLUSTER"</pre>	

Release Upgrade And Full Regression

Run from the repository on the Mac. This is the single maintained release-image workflow.

COMMAND	SELECT + COPY
<pre>cd /Users/sudheerkumar/Documents/Codex/2026-05-18/Mini-Call-Server export PLAYSBC_VERSION=2.6.0 kubectl config use-context kind-playsbc kubectl config set-context --current --namespace=playsbc helm upgrade --install playsbc \ "https://github.com/sudheerkumarvatrapu/PlaySBC/releases/download/v\${PLAYSBC_VERSION}/playsbc-\${PLAYSBC_VERSION}.tgz" \ --namespace playsbc \ --create-namespace \ --atomic \ --wait \ --timeout 10m \ -f configs/kubernetes/active-active-values.yaml \ --set image.repository=ghcr.io/sudheerkumarvatrapu/playsbc \ --set-string image.tag="\${PLAYSBC_VERSION}" \ --set image.pullPolicy=Always \ --set rtpengine.enabled=true \ --set rtpengine.image.repository=ghcr.io/sudheerkumarvatrapu/playsbc-rtpengine \ --set-string rtpengine.image.tag="\${PLAYSBC_VERSION}" \ --set rtpengine.image.pullPolicy=Always \ --set rtpengine.hostNetwork=false \ --set playsbc.config.media_backend=rtpengine \ --set-string playsbc.config.rtpengine_url=udp://playsbc-playsbc-rtpengine:2223 \ --set observability.enabled=true \ --set observability.prometheus.retention=31d \ --set observability.prometheus.persistence.size=5Gi \ --set observability.grafana.persistence.size=2Gi kubectl -n playsbc rollout status statefulset/playsbc-playsbc --timeout=240s kubectl -n playsbc rollout status statefulset/playsbc-playsbc-rtpengine --timeout=240s kubectl -n playsbc rollout status deployment/playsbc-playsbc-prometheus --timeout=240s kubectl -n playsbc rollout status deployment/playsbc-playsbc-grafana --timeout=240s kubectl -n playsbc get pods -o wide PYTHONPYCACHEPREFIX=/private/tmp/playsbc-pycache \ python3 tools/run_k8s_regression_job.py \ --all-profiles \ --runner-image "ghcr.io/sudheerkumarvatrapu/playsbc-k8s-regression:\${PLAYSBC_VERSION}" \ --sipp-image "ghcr.io/sudheerkumarvatrapu/playsbc-sipp:\${PLAYSBC_VERSION}" \ --playsbc-image "ghcr.io/sudheerkumarvatrapu/playsbc:\${PLAYSBC_VERSION}" \ --rtpengine-image "ghcr.io/sudheerkumarvatrapu/playsbc-rtpengine:\${PLAYSBC_VERSION}" \ --set-playsbc-image \ --set-rtpengine-image \ --no-load-playsbc-image \ --no-load-rtpengine-image \ --no-load-sipp-image \ --kind-cluster playsbc</pre>	

Outputs:

COMMAND	SELECT + COPY
<pre>logs/k8s-job/<run-id>/runner.log logs/k8s-job/<run-id>/k8s-reports/latest.html</pre>	

Build Current Source

Use this before publishing a release. It builds all images from the working tree, loads them into kind, and runs the full catalog.

COMMAND	SELECT + COPY
<pre>PYTHONPYCACHEPREFIX=/private/tmp/playsbc-pycache \ python3 tools/run_k8s_regression_job.py \ --all-profiles \ --build-playsbc-image \ --build-runner-image \ --build-sipp-image \ --build-rtpengine-image \ --kind-load-images \ --set-playsbc-image \ --set-rtpengine-image \ --kind-cluster playsbc</pre>	

This is the golden compatibility gate for local source changes.

Focused Runs

Rasa only:

COMMAND	SELECT + COPY
<pre>PYTHONPYCACHEPREFIX=/private/tmp/playsbc-pycache \ python3 tools/run_k8s_regression_job.py \ --rasa-profiles \ --build-playsbc-image \ --build-runner-image \ --build-sipp-image \ --kind-load-images \ --kind-cluster playsbc</pre>	

One or more named profiles:

COMMAND	SELECT + COPY
<pre>PYTHONPYCACHEPREFIX=/private/tmp/playsbc-pycache \ python3 tools/run_k8s_regression_job.py \ --profile ha-playsbc-midcall-failover \ --profile ha-rtengine-midcall-recovery \ --kind-cluster playsbc</pre>	

The next local HA milestone will add a dedicated shortcut for the complete multi-node HA catalog.

Observe A Run

COMMAND	SELECT + COPY
<pre>kubectl -n playsbc get job,pod -o wide kubectl -n playsbc get events --sort-by=.lastTimestamp tail -40</pre>	

Follow the newest runner:

COMMAND	SELECT + COPY
<pre>POD=\$(kubectl -n playsbc get pods \ -l app.kubernetes.io/name=playsbc-k8s-regression-runner \ --sort-by=.metadata.creationTimestamp \ -o jsonpath='{.items[-1:].metadata.name}') kubectl -n playsbc logs "\$POD" -c regression-runner -f</pre>	

Stop a run without deleting PlaySBC/RTPEngine:

COMMAND	SELECT + COPY
<pre>kubectl -n playsbc delete job \ -l app.kubernetes.io/name=playsbc-k8s-regression-runner \ --ignore-not-found kubectl -n playsbc delete pod \ -l app.kubernetes.io/name=playsbc-k8s-regression-runner \ --ignore-not-found</pre>	

Grafana And Prometheus

COMMAND

SELECT + COPY

```
kubectl -n playsbc port-forward svc/playsbc-playsbc-grafana 3000:3000
kubectl -n playsbc port-forward svc/playsbc-playsbc-prometheus 9090:9090
```

- Grafana: `http://127.0.0.1:3000`
- Prometheus: `http://127.0.0.1:9090`

See OBSERVABILITY.md for queries and dashboard interpretation.

Debug

COMMAND

SELECT + COPY

```
helm -n playsbc status playsbc
helm -n playsbc get values playsbc
helm -n playsbc history playsbc

kubectl -n playsbc get pods,svc,statefulset,deployment -o wide
kubectl -n playsbc get events --sort-by=.lastTimestamp | tail -60
kubectl -n playsbc logs statefulset/playsbc-playsbc --tail=120
kubectl -n playsbc logs statefulset/playsbc-playsbc-rtpengine --tail=120
```

Render the chart without changing the cluster:

COMMAND

SELECT + COPY

```
helm lint charts/playsbc
helm template playsbc charts/playsbc \
  --namespace playsbc \
  -f configs/kubernetes/active-active-values.yaml \
  --set observability.enabled=true >/tmp/playsbc-rendered.yaml
```

Cleanup

COMMAND

SELECT + COPY

```
helm -n playsbc uninstall playsbc
kubectl delete namespace playsbc --ignore-not-found
kind delete cluster --name playsbc
```

Rules

- Normal local regression uses active-active values and `rtpengine.hostNetwork=false`.
- Load profiles may skip PCAP; single-call profiles keep one combined PCAP and one `sipmsg.log`.
- Real secondary core/peer interfaces require Multus; default kind realms are logical.
- Do not run manual real-device calls while regression is mutating Helm profile values.

Restored Runbook: docs/KUBERNETES_LOCAL.md

Local Kubernetes Lab

This page explains the local topology. Use the [Kubernetes and Helm runbook](KUBERNETES_HELM_RUNBOOK.md) for maintained commands.

Default Topology

COMMAND	SELECT + COPY
<pre>kind (canonical) or minikube (compatibility) ■■■ PlaySBC-0 -> RTPengine-0 ■■■ PlaySBC-1 -> RTPengine-1 ■■■ shared HA lab state ■■■ Prometheus ■■■ Grafana ■■■ regression Job -> temporary SIPp core/peer pods</pre>	

Expected active-active pods:

COMMAND	SELECT + COPY
<pre>playsbc-playsbc-0 playsbc-playsbc-1 playsbc-playsbc-rtengine-0 playsbc-playsbc-rtengine-1</pre>	

Normal local deployment uses `configs/kubernetes/active-active-values.yaml`. RTPengine stays on pod networking because two replicas cannot bind the same host UDP range on a single kind node.

Realm Model

COMMAND	SELECT + COPY
<pre>Core realm: 172.28.0.0/24 Peer realm: 192.168.28.0/24</pre>	

In default kind/minikube these are logical realms represented in configuration, logs, reports, and metrics. Pods still receive normal CNI addresses such as `10.244.x.x`. Real secondary interfaces require Multus or another multi-network CNI.

kind And minikube

Use kind for the primary development and regression lane. The maintained cluster is named `playsbc`, and its `kubectl` context is `kind-playsbc`. kind runs every Kubernetes node as a Docker container, so Docker Desktop must be running before the cluster or its workloads are available.

Use minikube as a compatibility lane. It creates a separate cluster and context, usually named `minikube`; it is not part of a `kind-playsbc` deployment. Minikube's runtime requirement depends on its driver: `--driver=docker` requires Docker Desktop, while a VM-based driver requires its corresponding hypervisor instead. The same Helm chart and regression behavior must remain valid, but kind is the canonical command path.

Local cluster	kubectl context	Runtime dependency	PlaySBC role
kind playsbc	kind-playsbc	Docker Desktop	Primary development, HA, and full regression
minikube	minikube	Selected driver, commonly Docker Desktop	Compatibility validation

Stopping Docker Desktop stops access to the kind API server and pauses every PlaySBC, RTPengine, Grafana, and Prometheus container. Their Kubernetes objects remain present. Restarting Docker Desktop normally resumes the existing cluster; pod restart counts can increase, which is expected after the node runtime restarts.

Shared State

The current active-active lab uses stable StatefulSet identities and SQLite-backed shared registrar/dialog state. This is acceptable for a single-node experiment, not for production or robust multi-worker ownership.

The multi-node HA implementation will evaluate RWX storage and Redis/PostgreSQL so either worker can restore state safely.

Upcoming Multi-Node HA Lab

COMMAND	SELECT + COPY
<pre>control-plane ■■■ worker-1: PlaySBC-0 + RTPengine-0 ■■■ worker-2: PlaySBC-1 + RTPengine-1</pre>	

The next implementation adds:

- worker separation and pod anti-affinity
- PodDisruptionBudgets and node drain
- PlaySBC/RTPengine pair failure during active calls
- shared registrar/dialog restoration
- HA-specific Grafana panels and combined packet evidence

See [\[EVOLUTION_PLAN.md\]\(EVOLUTION_PLAN.md#next-implementation-local-multi-node-ha-lab\)](#).

Local Real Devices

OBI022 and Zoiper use the dedicated `playsbc-real-device` kind cluster when both are on the same LAN. This is intentionally separate from the active-active `playsbc` regression cluster and from AKS. The cluster exposes these ports one-to-one through kind `extraPortMappings`:

- SIP 5062/UDP, 5062/TCP, and 5061/TCP
- RTP/RTCP 30000-30049/UDP

PlaySBC and RTPengine advertise the Mac LAN IP. NodePort translation is not used for this RTP baseline. The chart rejects blank/mismatched LAN addresses, Azure exposure, active-active mode, or a media range other than 30000-30049 in this profile.

Use the maintained `[dedicated local real-device commands](KUBERNETES_HELM_RUNBOOK.md#dedicated-local-real-device-lab)`. This validates LAN device behavior, but not Azure LoadBalancer, public NAT, managed identity, or cloud firewall behavior.

Boundaries

- `kubectl port -forward` is suitable for HTTP, Grafana, Prometheus, and TCP checks; it does not solve UDP SIP/RTP exposure.
- Multi-node kind still runs on one Mac and cannot prove physical host or availability-zone failure.
- AKS remains the milestone lane for Azure identity, ACR, public LoadBalancers, static IPs, and internet real-device calls.

-
- The local real-device capture command must include `--context kind-playsbc-real-device`; the AKS capture must use its AKS context. Context isolation is part of the evidence contract.

Restored Runbook: docs/OBSERVABILITY.md

PlaySBC Observability

The lab observability path is:

COMMAND	SELECT + COPY
<code>PlaySBC /metrics -> Prometheus -> Grafana</code>	

PlaySBC also exports RTPengine and AI/Rasa state derived from its call-control evidence. Enable or upgrade the stack with the canonical command in KUBERNETES_HELM_RUNBOOK.md.

What Is Measured

Area	Examples
Calls	active, admitted, completed, rejected, peak
SIP	requests and responses by realm, method, direction, status, and class
Media	negotiated codecs, transcoding intent, active RTPengine sessions, failures
HA	node health, drain state, shared registrations, shared dialogs
AI	STT, Rasa, TTS, prompt, fallback, and bot-action counters

Prometheus defaults to a short scrape interval for brief SIPp calls and 31-day retention. Persistence depends on a working cluster storage class.

Open Grafana

COMMAND	SELECT + COPY
<code>kubectl -n playsbc port-forward svc/playsbc-playsbc-grafana 3000:3000</code>	

Open `http://127.0.0.1:3000` and use the lab credentials:

COMMAND	SELECT + COPY
<pre>user: admin password: playsbc-lab dashboard: PlaySBC Core/Peer SBC Lab</pre>	

Query Prometheus

COMMAND	SELECT + COPY
<code>kubectl -n playsbc port-forward svc/playsbc-playsbc-prometheus 9090:9090</code>	

Open `http://127.0.0.1:9090`. Useful queries:

COMMAND	SELECT + COPY
<pre> sum(playsbc_active_calls) sum(increase(playsbc_b2bua_calls_total[15m])) sum(increase(playsbc_b2bua_calls_completed_total[15m])) sum by (realm,method,direction) (increase(playsbc_sip_requests_total[15m])) sum by (realm,status,status_class,direction) (increase(playsbc_sip_responses_total[15m])) sum by (realm,trunk) (max_over_time(playsbc_trunk_healthy[15m])) sum by (backend,inbound_codec,outbound_codec,transcoding) (increase(playsbc_media_negotiations_total[15m])) sum by (backend,inbound_codec,outbound_codec) (increase(playsbc_transcoding_sessions_total[15m])) sum by (from_realm,to_realm) (playsbc_rtpengine_media_sessions_active) sum(increase(playsbc_rtpengine_control_failures_total[15m])) sum by (cluster,node) (playsbc_ha_shared_registrations) sum by (cluster,node) (playsbc_ha_shared_dialogs) sum by (cluster,node) (playsbc_ha_node_draining) sum by (bot,stt,tts) (increase(playsbc_ai_voice_turns_total[15m])) sum(increase(playsbc_ai_rasa_failures_total[15m])) </pre>	

Interpret The Panels

- Counters reset when regression rolls PlaySBC. Use `increase(metric[window])`.
- `playsbc_active_calls` is a live gauge and should return to 0 after calls end.
- Range panels intentionally retain completed calls until the selected time window moves forward.
- Use `max_over_time` only for gauges such as peak calls or trunk health.
- A value such as 2.1 on a smoothed panel is a rate or average, not a fractional call.
- Grafana and Prometheus should match because Grafana queries Prometheus; compare the exact query, time range, job filter, and refresh interval.

Direct Metrics Check

COMMAND	SELECT + COPY
<pre> kubectl -n playsbc port-forward svc/playsbc-playsbc 8080:8080 curl http://127.0.0.1:8080/metrics </pre>	

The endpoint returns Prometheus text with `# HELP`, `# TYPE`, and labels such as `cluster`, `node`, `realm`, `trunk`, `backend`, `inbound_codec`, `outbound_codec`, and `transcoding`.

Optional Operator Resources

For clusters with Prometheus Operator CRDs:

COMMAND	SELECT + COPY
<pre> helm upgrade playsbc charts/playsbc \ --namespace playsbc \ --reuse-values \ --set observability.prometheus.serviceMonitor.enabled=true \ --set observability.prometheus.rules.enabled=true </pre>	

Current Boundary

RTPengine does not expose native Prometheus metrics through this chart. PlaySBC reports RTPengine control failures, call-owned media sessions, codec negotiation, and transcoding intent from its own state. Packet-level truth remains in RTPengine query evidence and `capture.pcap`.

Restored Runbook: docs/README.md

PlaySBC Documentation

Use one canonical page for each task. Supporting pages explain design and evidence; they do not repeat full deployment commands.

Choose A Workflow

Task	Canonical Page	Environment
Local release deployment and full regression	[Kubernetes and Helm runbook](KUBERNETES_HELM_RUNBOOK.md)	kind/minikube
Local source-image compatibility gate	[Kubernetes and Helm runbook](KUBERNETES_HELM_RUNBOOK.md#build-current-source)	kind
Local topology and networking	[Local Kubernetes lab](KUBERNETES_LOCAL.md)	kind/minikube
OBi1022/Zoiper local LAN calls	[Kubernetes and Helm runbook](KUBERNETES_HELM_RUNBOOK.md#dedicated-local-real-device-lab)	dedicated kind
Azure creation, deployment, regression, and cleanup	[Azure AKS runbook](AZURE_AKS.md)	AKS/Cloud Shell
OBi1022 and Zoiper calls	[Real-device lab](REAL_DEVICE_LAB.md)	AKS or dedicated kind
RTPengine design and focused checks	[RTPengine](RTPENGINE_LOCAL.md)	local/Kubernetes
Rasa voice and chat regression	[AI Voice Gateway](AI_VOICE_GATEWAY.md)	Docker/Kubernetes
Grafana, Prometheus, and metrics	[Observability](OBSERVABILITY.md)	Kubernetes
Roadmap and production gates	[Evolution plan](EVOLUTION_PLAN.md)	all
Release assets and historical notes	[Release index](../release/README.md)	GitHub

Default Test Strategy

Lane	Purpose	Frequency
Docker dual-realm	Fast signalling/media regression	Every feature
Local kind	Full suite, active-active, HA, failover, observability	Every feature/release
AKS	Azure LoadBalancer, ACR, public SIP/RTP, real-device smoke	Release milestones
Real device	OBi1022/Zoiper signalling and two-way media	Media/release milestones

Documentation Rules

1. The current version appears in the README, canonical runbooks, chart metadata, and release index.
2. A command has one canonical home. Other pages link to it.
3. Release notes are historical and are not rewritten during documentation cleanup.
4. Commands must fail before changing workloads when variables or images are invalid.
5. Local kind regression, AKS regression, Docker regression, and real-device behavior must remain isolated.

Current release: v2.6.0.

Restored Runbook: docs/REAL_DEVICE_LAB.md

PlaySBC Real-Device Lab

This guide covers the validated OBi1022 1001 and Zoiper 1002 call flow in AKS and the local LAN lane. Build the AKS base with AZURE_AKS.md, or use the separate [kind real-device procedure](KUBERNETES_HELM_RUNBOOK.md#dedicated-local-real-device-lab).

COMMAND	SELECT + COPY
<pre> Obi1022 1001 -> Internet/NAT -> Azure SIP LB -> PlaySBC -> RTPengine -> Azure RTP LB -> Internet/NAT -> Zoiper 1002 </pre>	

Validated baseline: SIP UDP 5062, RTP/RTCP UDP 30000-30049, PCMU/PCMA, digest REGISTER, and two-way RTPengine-anchored audio.

Lane	SIP/RTP advertised address	Kube context	What it proves
AKS	Azure SIP and RTP public IPs	AKS context	Public LB, NAT, ACR, internet devices
Local kind	Mac LAN IPv4	kind-playsbc-real-device	LAN registration, calls, media, capture

Do not reuse the local values file in AKS or the AKS values file locally. The capture tool accepts `--context` so evidence collection cannot silently follow whichever context was selected last.

1. Apply Real-Device Values

Run in Cloud Shell after the base AKS services have public IPs:

COMMAND	SELECT + COPY
<pre> export PLAYSBC_VERSION=2.6.0 export AKS_RG=playsbc-aks-rg export NETWORK_RG=playsbc-network-rg export AKS_NAME=playsbc-aks export SIP_PIP_NAME=playsbc-sip-pip export RTP_PIP_NAME=playsbc-rtp-pip export ACR_NAME=\$(az acr list --resource-group "\$AKS_RG" --query '[0].name' -o tsv) export ACR_LOGIN_SERVER=\$(az acr show -g "\$AKS_RG" -n "\$ACR_NAME" --query loginServer -o tsv) export SIP_PUBLIC_IP=\$(az network public-ip show -g "\$NETWORK_RG" -n "\$SIP_PIP_NAME" --query ipAddress -o tsv) export RTP_PUBLIC_IP=\$(az network public-ip show -g "\$NETWORK_RG" -n "\$RTP_PIP_NAME" --query ipAddress -o tsv) helm upgrade playsbc \ "https://github.com/sudheerkumarvatrapu/PlaySBC/releases/download/v\${PLAYSBC_VERSION}/playsbc-\${PLAYSBC_VERSION}.tgz" \ --namespace playsbc \ --reuse-values \ --atomic \ --wait \ --timeout 10m \ --set image.repository="\$ACR_LOGIN_SERVER/playsbc" \ --set-string image.tag="\$PLAYSBC_VERSION" \ --set rtpengine.image.repository="\$ACR_LOGIN_SERVER/playsbc-rtpengine" \ </pre>	

COMMAND (CONTINUED)	SELECT + COPY
<pre>--set-string rtpengine.image.tag="\$PLAYSBC_VERSION" \ --set rtpengine.rtpMin=30000 \ --set rtpengine.rtpMax=30049 \ --set-string rtpengine.advertisedIP="\$RTP_PUBLIC_IP" \ --set playsbc.config.rtp_min=30000 \ --set playsbc.config.rtp_max=30049 \ --set playsbc.config.media_backend=rtpengine \ --set-string playsbc.config.rtpengine_url=udp://playsbc-playsbc-rtpengine:2223 \ --set playsbc.config.reject_unknown_routes=true \ --set playsbc.config.b2bua_invite_timeout=60.0 \ --set playsbc.config.rtpengine_g711_only=true \ --set playsbc.config.rtpengine_plain_rtp_sdp=true \ --set playsbc.config.rtpengine_explicit_rtcp=true \ --set playsbc.config.rtpengine_sip_source_address=true \ --set playsbc.config.rtpengine_media_handover=true \ --set playsbc.config.rtpengine_nat_wait=true \ --set playsbc.config.rtpengine_pierce_nat=false \ --set-string playsbc.config.sip_advertised_ip="\$SIP_PUBLIC_IP" \ --set-string playsbc.config.b2bua_advertised_ip="\$SIP_PUBLIC_IP" \ --set authSecret.enabled=true \ --set-string authSecret.users.1001=secret-password \ --set-string authSecret.users.1002=secret-password</pre>	
COMMAND (CONTINUED)	SELECT + COPY
<pre>kubectl -n playsbc rollout status deployment/playsbc-playsbc --timeout=240s kubectl -n playsbc rollout status deployment/playsbc-playsbc-rtpengine --timeout=240s kubectl -n playsbc get pods -o wide kubectl -n playsbc get svc playsbc-playsbc-azure-sip-public playsbc-playsbc-azure-rtp-public -o wide</pre>	

The SIP service must show SIP_PUBLIC_IP; the RTP service and RTPengine command must show RTP_PUBLIC_IP and 30000-30049.

2. Configure OBi1022 As 1001

Open <http://192.168.1.9> and disable old provider provisioning:

COMMAND	SELECT + COPY
<pre>System Management -> Auto Provisioning OBiTALK Provisioning: Disabled ITSP Provisioning: Disabled Firmware Update: Manual or Disabled</pre>	

Configure SP1:

COMMAND	SELECT + COPY
<pre>Service Providers -> ITSP Profile A -> SIP ProxyServer: <SIP_PUBLIC_IP> ProxyServerPort: 5062 ProxyServerTransport: UDP RegistrarServer: <SIP_PUBLIC_IP> RegistrarServerPort: 5062 OutboundProxy: blank X_DnsSrv: false X_UseTokenAuth: false X_DiscoverPublicAddress: true X_UsePublicAddressInVia: true X_UseRport: true Voice Services -> SP1 Service Enable: checked X_ServProvProfile: A AuthUserName: 1001 AuthPassword: secret-password URI: 1001 RegisterEnable: checked KeepAliveEnable: checked</pre>	
COMMAND (CONTINUED)	SELECT + COPY
<pre>Service Providers -> ITSP Profile A -> RTP X_RTPTransport: UDP</pre>	

Submit and reboot the phone.

3. Configure Zoiper As 1002

COMMAND	SELECT + COPY
<pre>Username: 1002 Password: secret-password Domain / Host: <SIP_PUBLIC_IP>:5062 Transport: UDP Outbound proxy: blank</pre>	

4. Monitor Signalling And Media

Registration only:

COMMAND	SELECT + COPY
<pre>kubectl -n playsbc logs deployment/playsbc-playsbc -f --since=10m \ grep -aE 'REGISTER Registered 401 403 1001 1002'</pre>	

Calls and media from both PlaySBC and RTPengine:

COMMAND	SELECT + COPY
<pre>kubectl -n playsbc logs -f \ -l app.kubernetes.io/instance=playsbc \ --all-containers=true \ --prefix \ --max-log-requests=10 \ --since=10m \ grep -aE 'SIP (INVITE ACK BYE CANCEL) SIP TX response SIP response INVITE ROUTE B2BUA SDP SUMMARY RTPENGINE RTPengine RTP packet RTCP 1001 1002'</pre>	

Place both calls:

COMMAND	SELECT + COPY
<pre>0Bi1022 1001 -> Zoiper 1002 Zoiper 1002 -> 0Bi1022 1001</pre>	

Pass criteria:

- both devices register after a 401 challenge
- INVITE routes to the registered peer
- INVITE/100/180/200/ACK and BYE/200 complete
- SDP advertises the configured RTP public/LAN IP and ports inside 30000-30049
- RTPengine reports caller_to_callee=observed and callee_to_caller=observed
- audio is heard in both directions

5. Capture One Evidence Archive

Use the released capture tool and one temporary host-network netshoot pod:

COMMAND

SELECT + COPY

```
export PLAYSBC_VERSION=2.6.0

export AKS_CONTEXT=$(kubectl config current-context)

cd ~
rm -rf "PlaySBC-v$PLAYSBC_VERSION"
git clone --branch "v$PLAYSBC_VERSION" --depth 1 \
  https://github.com/sudheerkumarvatrapu/PlaySBC.git \
  "PlaySBC-v$PLAYSBC_VERSION"
cd "PlaySBC-v$PLAYSBC_VERSION"

PYTHONPYCACHEPREFIX=/tmp/playsbc-pycache \
python3 tools/run_real_device_capture.py \
  --context "$AKS_CONTEXT" \
  --namespace playsbc \
  --duration 120 \
  --capture-image nicolaka/netshoot:latest
```

If Docker Hub pulls are blocked:

COMMAND

SELECT + COPY

```
az acr import \
  --name "$ACR_NAME" \
  --source docker.io/nicolaka/netshoot:latest \
  --image netshoot:latest \
  --force

PYTHONPYCACHEPREFIX=/tmp/playsbc-pycache \
python3 tools/run_real_device_capture.py \
  --context "$AKS_CONTEXT" \
  --namespace playsbc \
  --duration 120 \
  --capture-image "$ACR_LOGIN_SERVER/netshoot:latest"
```

Press **Ctrl-C** once when both calls finish. The tool saves evidence before deleting the capture pod.

Output:

COMMAND

SELECT + COPY

```
logs/Real-Device-Lab/real-device-capture-<timestamp>/
capture.pcap
sipmsg.log
canonical-sip.json
media-evidence.log
media-evidence.json
latest.html
playsbc.log
rtengine.log
rtengine-verdict.log
summary.log

logs/Real-Device-Lab/real-device-capture-<timestamp>.tgz
```

Download the .tgz immediately from Cloud Shell.

6. Interpret Evidence

- capture.pcap is the only raw packet capture.
- sipmsg.log collapses near-simultaneous host/LB/pod mirrors into one call ladder.
- Later repeated SIP-over-UDP messages remain as retransmission annotations.
- media-evidence.log separates PCMU/PCMA speech, RTCP, telephone events, and tiny NAT probes.
- Host-network capture counts are observations because one packet may appear on several interfaces. RTPengine query totals are authoritative for unique session packet counts.
- One-sided RTCP is reported as endpoint - limited; it does not invalidate proven two-way RTP.

7. Fast Troubleshooting

Symptom	Check
No REGISTER	SIP public IP, UDP 5062, home SIP ALG, OBi provisioning
Repeated 401	User/password, token auth, realm
404/address incomplete	INVITE ROUTE SELECTED; both users must be registered
Unparsable SDP	Unknown-route fallback must remain disabled
480	Answer before the 60-second B2BUA timeout
One-way/no audio	RTP LB IP, advertised IP, 30000-30049, NAT learning flags
RTP after 180	Check whether packets are tiny NAT probes or real G.711 speech
Call remains connected	Verify BYE/200 on both B2BUA legs and RTPengine delete
Capture pod remains	Delete pod/real-device-capture-* - pod after confirming evidence copied

Basic snapshot:

COMMAND	SELECT + COPY
<pre>kubectl -n playsbc get pods,svc -o wide kubectl -n playsbc logs deployment/playsbc-playsbc --tail=200 kubectl -n playsbc logs deployment/playsbc-playsbc-rtpengine --tail=200</pre>	

8. Validated Result And Next Work

The 2026-08-23 v2.5.0 run passed OBi1022-to-Zoiper and Zoiper-to-OBi1022 signalling, two-way PCMU, normal teardown, combined PCAP, and archive generation. Zoiper emitted RTCP receiver reports; OBi1022 did not, so RTCP was correctly endpoint-limited.

Next real-device work:

- repeatable local-LAN testing through the dedicated kind cluster
- Poly VVX600 and Yealink SIP-T33G TCP/TLS registration
- longer calls, re-registration, multiple devices, and SIP ALG detection
- richer RTCP loss/jitter and MOS-style evidence
- local PlaySBC/RTPengine HA and failover during device calls

Restored Runbook: docs/RTPENGINE_LOCAL.md

RTPengine For PlaySBC

PlaySBC owns SIP/B2BUA control. Sipwise RTPengine anchors RTP/RTCP and performs media transformation.

COMMAND	SELECT + COPY
<pre>SIP: endpoint A <-> PlaySBC <-> endpoint B RTP: endpoint A <-> RTPengine <-> endpoint B</pre>	

Supported Lab Models

Model	Use
Docker regression	Dual-realm SIPp media and fault profiles
Kubernetes active-active	Paired PlaySBC and RTPengine replicas
AKS real-device	Public SIP LB plus public RTP LB on UDP 30000-30049
Standalone container	Local RTPengine control development

Use KUBERNETES_HELM_RUNBOOK.md for Kubernetes commands and AZURE_AKS.md for public Azure media exposure.

Docker Regression

COMMAND	SELECT + COPY
<pre>PYTHONPYCACHEPREFIX=/private/tmp/playsbc-pycache \ python3 tools/run_regression_suite.py \ --skip-sipp-smoke \ --all-b2bua-profiles \ --timeout 420</pre>	

Realm	SIPp	PlaySBC	RTPengine
Core	172.28.0.10	172.28.0.20	172.28.0.40
Peer	192.168.28.30	192.168.28.20	192.168.28.40

The suite renders each profile, isolates core and peer networks, captures one merged PCAP, and validates RTPengine control, codec negotiation, transcoding, secure-media interworking, and fault behavior.

Active-Active Pairing

Each PlaySBC node selects its paired RTPengine from `ha.rtpengine_pairs`. New calls can be drained from a node while existing dialog cleanup remains allowed.

COMMAND	SELECT + COPY
<pre>ha: enabled: true cluster_id: playsbc-aa-lab nodes: - node_id: playsbc-0 state: active - node_id: playsbc-1 state: active rtpengine_pairs: - node_id: playsbc-0 rtpengine_url: udp://playsbc-rtpengine-0:2223 - node_id: playsbc-1 rtpengine_url: udp://playsbc-rtpengine-1:2223</pre>	

Shared SQLite is a lab mechanism, not a production-grade cross-node state store. The multi-node roadmap is in EVOLUTION_PLAN.md.

Standalone Container

COMMAND	SELECT + COPY
<pre> docker build -f docker/rtpengine.Dockerfile -t playsbc/rtpengine:local . docker rm -f playsbc-rtpengine 2>/dev/null true docker run -d --name playsbc-rtpengine \ -p 2223:2223/udp \ -p 30000-32000:30000-32000/udp \ playsbc/rtpengine:local python3 tools/check_rtpengine.py \ --url udp://127.0.0.1:2223 \ --timeout 1 </pre>	

Expected result: RTPengine OK ... result=pong.

Evidence And Diagnosis

Artifact	Meaning
log.media	offer, answer, query, endpoint learning, and packets
log.transcoding	codec path and transcoding owner
sipmsg.log	normalized SIP messages
capture.pcap	combined signalling, RTP, RTCP, and networking evidence
latest.html	profile ladder and final verdict

For standalone failures, run `docker logs --tail 100 playsbc-rtpengine` and repeat the readiness check. For AKS one-way audio, verify the advertised RTP IP, the exact 30000-30049 range on both RTPengine and the Azure LoadBalancer, and learned packet counts in both directions.

AKS Complete Cleanup - One Command

Use this when the entire PlaySBC Azure lab must be removed before a fresh start. It deletes `playsbc-aks-rg` and `playsbc-network-rg`, waits for both asynchronous deletions, and then removes the stale kubeconfig entries. The AKS-managed `MC_*` resource group is removed with the AKS resource group; do not delete it separately.

COMMAND	SELECT + COPY
<pre> bash -lc ' set -euo pipefail AKS_RG=playsbc-aks-rg NETWORK_RG=playsbc-network-rg AKS_NAME=playsbc-aks SUB_ID=\$(az account show --query id -o tsv) : "\${SUB_ID:?No active Azure subscription}" az account show \ --query "{Subscription:name,ID:id,Tenant:tenantId}" \ -o table az group list \ --subscription "\$SUB_ID" \ --query "[?name==`\$AKS_RG` name==`\$NETWORK_RG`].{Name:name,Location:location,State:properties.provisioningState}" \ -o table read -r -p "Type DELETE to permanently remove all PlaySBC Azure resources: " CONFIRM ["\$CONFIRM" = DELETE] { echo "Cancelled."; exit 1; } pkill -INT -f "helm upgrade.*playsbc" 2>/dev/null true </pre>	
COMMAND (CONTINUED)	SELECT + COPY
<pre> for RG in "\$AKS_RG" "\$NETWORK_RG"; do if ["\$(az group exists --subscription "\$SUB_ID" --name "\$RG")" = true]; then az group delete \ --subscription "\$SUB_ID" \ --name "\$RG" \ --yes \ --no-wait fi done while true; do A=\$(az group exists --subscription "\$SUB_ID" --name "\$AKS_RG") N=\$(az group exists --subscription "\$SUB_ID" --name "\$NETWORK_RG") echo "\$(date) \$AKS_RG=\$A \$NETWORK_RG=\$N" ["\$A" = false] && ["\$N" = false] && break sleep 30 done kubectl config delete-context "\$AKS_NAME" 2>/dev/null true kubectl config delete-cluster "\$AKS_NAME" 2>/dev/null true kubectl config unset "users.clusterUser_\${AKS_RG}_\${AKS_NAME}" 2>/dev/null true </pre>	
COMMAND (CONTINUED)	SELECT + COPY
<pre> echo "All PlaySBC Azure resources deleted." </pre>	